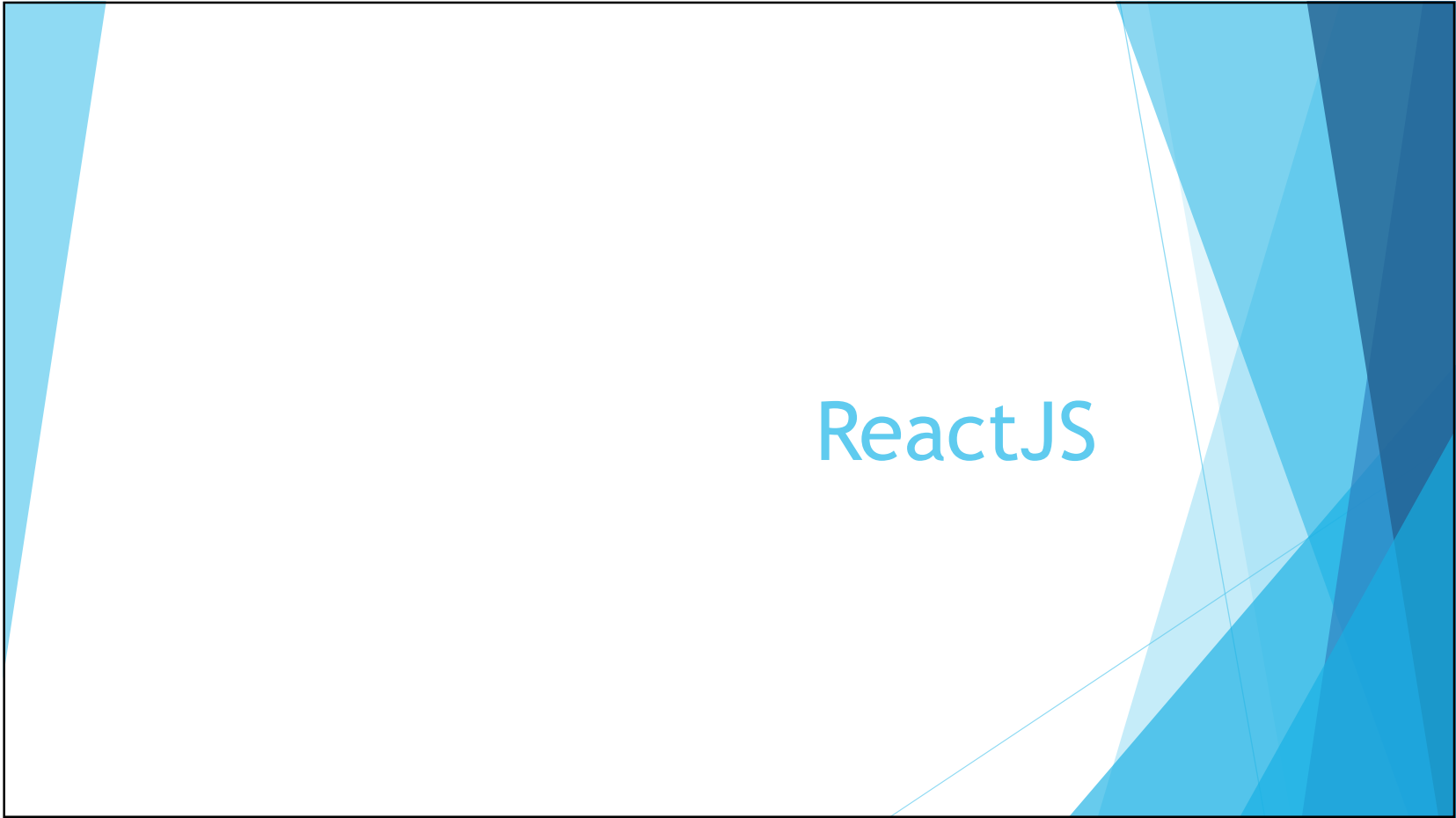


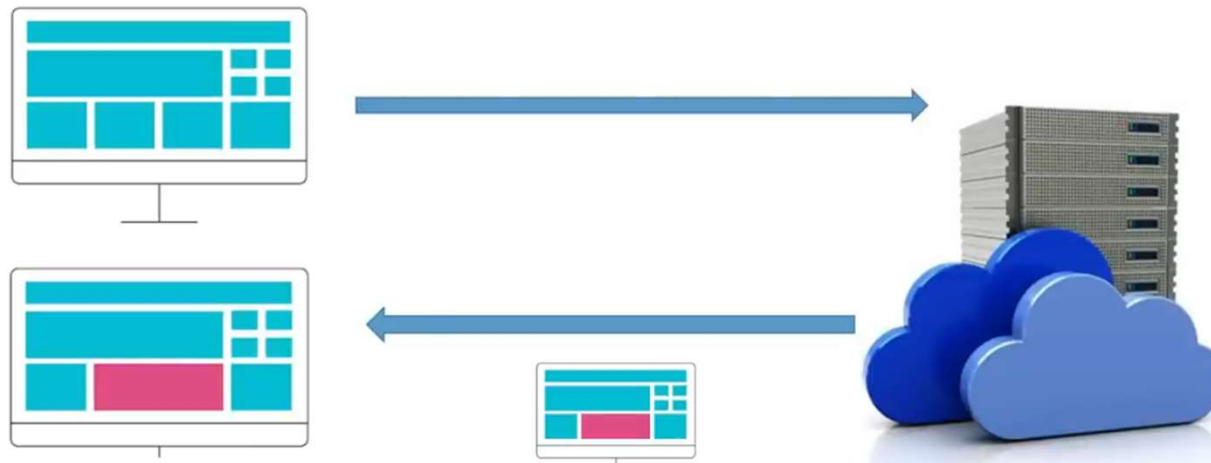
The ReactJS logo is centered on a white background. It features the text "ReactJS" in a blue, sans-serif font. The background is decorated with abstract, overlapping geometric shapes in various shades of blue, including light blue, medium blue, and dark blue, creating a modern and dynamic feel.

ReactJS

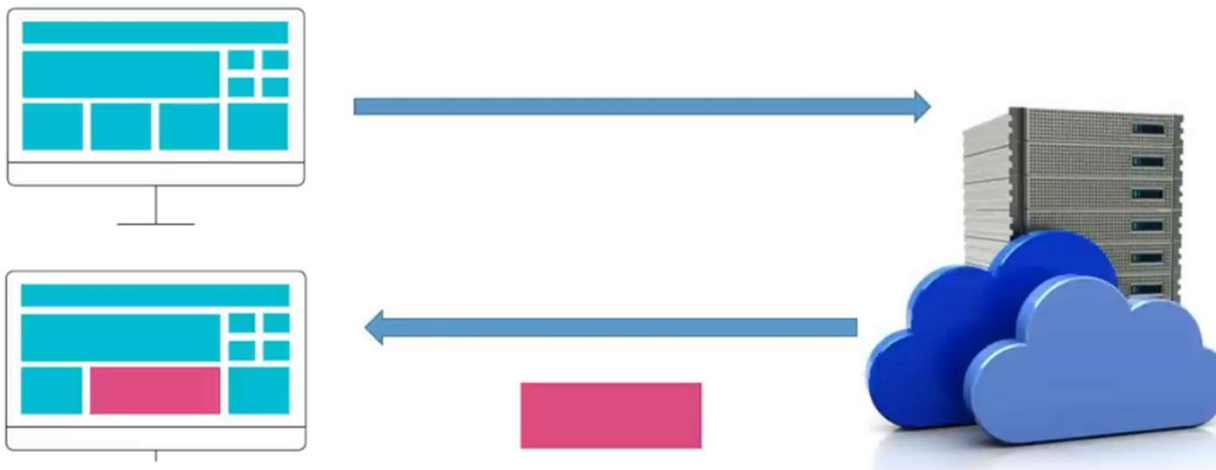
What is REACT

- ▶ 1. Javascript library to build **Dynamic** and **interactive** user interfaces
- ▶ 2. developed at facebook in 2011
- ▶ 3. Currently most widely used JS library for front-end development
- ▶ 4. Used to create single page application

Multi Page Application (Normal Website)



Single Page Application (SPA)



Working of DOM

- ▶ 1. Browser takes HTML and create **DOM**
- ▶ 2. JS helps us modify DOM based on user actions or events
- ▶ 3. In big applications, working with DOM becomes complicated

Problems with JS

- ▶ 1.React has a simpler mental model
- ▶ 2.JS is cumbersome
- ▶ 3.JS is error-prone
- ▶ 4.JS is hard to maintain

Working with REACT

- ▶ 1. No need to worry about **querying** and **updating** DOM
- ▶ 2. React creates a webpage with **small** and **reusable** components
- ▶ 3. React will take care of **creating** and **updating** DOM elements
- ▶ 4. It saves a lot of time

JS Vs React

- ▶ JS is imperative: You define steps to reach your desire state
- ▶ React is declarative : You define the target UI state and then React figures out how to reach that state.

Introduction to Components

- ▶ React application is a tree of components with App Component as the root bringing everything together.

Install Latest Node

- ▶ 1. Search Download NodeJS-- Node is working as a light server for react

Create a React App

- ▶ 1. Official tool is CRA (Create React APP)
 - ▶ 2. Vite is a modern tool to create React Project
 - ▶ 3. Vite produces Quick and Small bundle size
 - ▶ 4. Vite: Use `npm run dev` to launch dev server
 - ▶ 5. Use `npm start` for CRA
-
- ▶ on terminal
-
- ▶ `npm create vite@latest`

Project Structure

- ▶ `node_modules`: dependencies are installed as node packages
- ▶ `public/` directory: contains static files that don't change
- ▶ `src/` Directory: main folder for React code
 - ▶ 1. `components/`: Reusable parts of the UI , like buttons or headers.
 - ▶ 2. `assets/`: images , fonts and other static files
 - ▶ 3. `styles/`: CSS or stylesheets
- ▶ `package.json` contains information about this project like name , version , dependencies on other react packages.
- ▶ `vite.config.js` contains vite config
- ▶ `package-lock.json`-- maintains the current values of various dependencies used in the project

Our First App

- ▶ Component is a simple function

- ▶ ./App.jsx

```
function App(){
```

```
  return <h1>Hello World </h1>
```

```
}
```

```
export default App
```

Types of Components

▶ Class Components:

- 1.Stateful: Can manage State
2. Lifecycle: Access to lifecycle methods
3. Verbose: More boilerplate code
4. Not preferred anymore

Functional Components:

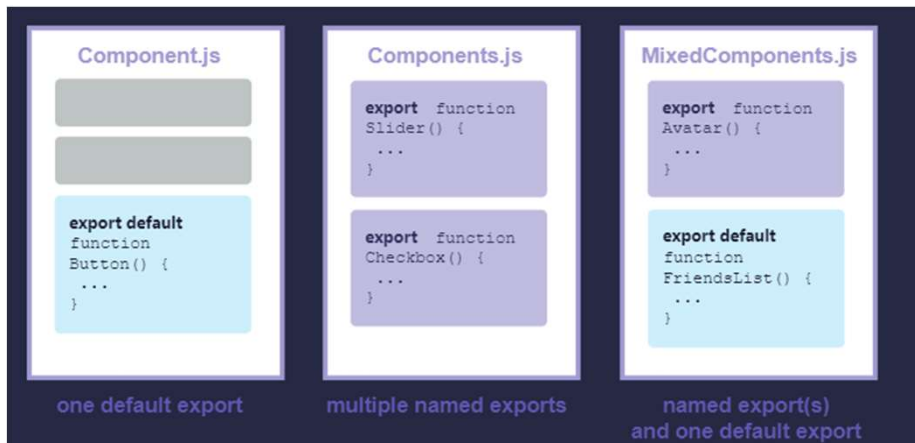
1. Initially Stateless
2. Can use Hooks for state and effects
3. Simpler and more concise
4. More Popular

Importing and Exporting a component

- ▶ You cannot place all components in one file or root component file. You need to move various components out of root component file.
- ▶ This will make them more modular and reusable in other files
- ▶ You can move a component in three steps:
 1. **Make** a new JS file to put the components in.
 2. **Export** your function component from that file (using either [default](#) or [named](#) exports).
 3. **Import** it in the file where you'll use the component (using the corresponding technique for importing [default](#) or [named](#) exports).

Importing and Exporting components

- ▶ There are two primary ways to export values with JavaScript: default exports and named exports.
- ▶ A file can have no more than one *default* export, but it can have as many *named* exports as you like.



Importing and Exporting components

Syntax	Export statement	Import statement
Default	<pre>export default function Button() {}</pre>	<pre>import Button from './Button.js';</pre>
Named	<pre>export function Button() {}</pre>	<pre>import { Button } from './Button.js';</pre>

JSX

- ▶ **JSX is a syntax extension for JavaScript that lets you write HTML-like markup inside a JavaScript file.**
- ▶ **In React, rendering logic and markup live together in the same place—components.**
- ▶ JSX determines how the UI will look wherever the component is used.
- ▶ JSX stands for Javascript XML
- ▶ JSX gets converted to regular JavaScript

```
Sidebar() {  
  if (isLoggedIn()) {  
    <p>Welcome</p>  
  } else {  
    <Form />  
  }  
}
```

Sidebar.js React component

```
Form() {  
  onClick() {...}  
  onSubmit() {...}  
  
  <form onSubmit>  
    <input onClick />  
    <input onClick />  
  </form>  
}
```

Form.js React component